

# Repository in one sentence

This repository is an **explainable admission and scheduling policy for SAGE/Waggle edge applications**.

It decides **which ready application should run next**. It does not itself run AI models, control Kubernetes Pods, implement MQTT/Meshtastic, or manage model degradation.

The documented end-to-end boundary is:

```
SAGE science rule
-> ready plugin queue
-> this repository ranks and selects plugins
-> SAGE NodeScheduler creates the Pod
-> Kubernetes chooses the node
```

See **architecture.md**.

## 1. The core scheduling engine

**What already exists:**

- A generic Task representation containing identity, priority, enqueue time, estimated runtime, deadline, GPU requirement, predicted success rate, and CPU/memory/GPU resources.
- A Snapshot containing ready tasks, running tasks, the decision time, and optional available capacity.
- An explainable Decision describing every candidate.

See **model.go**.

The engine has three possible outcomes:

- **selected**: the task can start now.
- **deferred**: the task is valid but cannot start yet.
- **rejected**: its identity or metadata is invalid.

Every outcome includes a reason such as:

- `concurrency_limit`
- `gpu_capacity`
- `resource_capacity`
- `reliability_threshold`
- `already_running`
- `invalid_task`

See **engine.go**.

## How ranking works

For every ready task, the engine calculates:

```
deadline = explicit deadline
           or enqueue time + maximum queue latency
```

```
slack = deadline - current time - estimated runtime
```

It then creates four normalized scores:

- Priority
- Deadline urgency through slack
- Time already spent in the queue
- Predicted probability of success

The example policy weights them as:

```
45% priority
30% deadline slack
15% queue age
10% reliability
```

See **engine.go** and **policy.example.yaml**.

An older task gains age points, which reduces permanent starvation. A task close to or beyond its deadline gains urgency. An impossible deadline is reported through `predictedDeadlineMiss`, but the task may still be selected because the engine does not know an alternative application.

When scores are equal, selection remains deterministic:

1. Earliest deadline
2. Oldest enqueue time
3. Lexical task ID

## How admission works

After ranking, the engine checks:

1. Is the task valid and unique?
2. Is it already running?
3. Does it meet the minimum success-rate threshold?
4. Is there a global concurrency slot?
5. Is there a GPU concurrency slot?
6. Does it fit available resources, if resource fitting is enabled?

The relevant selection loop is in **engine.go**.

## 2. Policy configuration

**What already exists:**

- Strict, versioned YAML configuration.
- Global and GPU concurrency limits.
- Default priority, runtime, latency, and success rate.
- Minimum acceptable success rate.
- Configurable ranking weights.
- Optional resource-fit enforcement.
- Fail-open behavior.
- A security switch controlling whether applications may influence scheduling through environment variables.

See **config.go**.

The loader rejects:

- Unknown YAML fields
- Missing schema headers
- Multiple YAML documents
- Invalid durations
- Invalid probabilities
- Invalid or unbounded concurrency
- Negative or non-finite weights

The supplied example is intentionally conservative:

- One application at a time
- One GPU application at a time
- Minimum success rate of 0.50
- Resource fitting disabled
- Workload hints disabled
- Fail-open enabled

A significant detail: because `trustWorkloadEnvHints` is `false`, application-provided priority, runtime, deadline, and success hints are ignored by default. Every workload receives the policy defaults until an operator explicitly enables those hints.

### 3. The SAGE/Waggle adapter

The pure engine has no SAGE or Kubernetes dependency. The adapter translates between Waggle objects and the engine.

**What already exists:**

- An implementation of Waggle's compiled `SchedulingPolicy` interface.
- Conversion from `PluginRuntime` to the generic `Task` structure.
- Snapshotting of the ready and scheduled queues.
- Validation of Kubernetes names and resource quantities.
- Preservation of the original queue pointers when returning selected plugins.
- In-memory tracking of when each task was first observed.
- A stored copy of the latest explained decision.

See [\*\*policy.go\*\*](#).

The adapter does **not** modify the queues. It only returns its selection. The existing Waggle `NodeScheduler` remains responsible for:

- Removing selected plugins from the ready queue
- Moving them to the scheduled queue
- Creating Pods
- Tracking their lifecycle

## Workload scheduling hints

The adapter recognizes:

- SAGE\_SCHEDULER\_PRIORITY
- SAGE\_SCHEDULER\_MAX\_LATENCY
- SAGE\_SCHEDULER\_ESTIMATED\_RUNTIME
- SAGE\_SCHEDULER\_DEADLINE\_AT
- SAGE\_SCHEDULER\_SUCCESS\_RATE

See **hints.go**.

These values are declarations from the application author - not measurements made by the scheduler. In particular, the "success rate" is not currently learned from inference results or operational telemetry.

## GPU and resource handling

A plugin is classified as GPU work through:

- `selector.resource.gpu`
- `limit.gpu`
- `nvidia.com/gpu`

CPU, memory, GPU, and extended-resource quantities are validated. Requests cannot exceed their corresponding limits.

However, real resource fitting is disabled by default because the pinned Waggle version supplies fictitious availability. Kubernetes still performs the final real resource enforcement.

## 4. Failure handling

**What already exists:**

- Invalid tasks are rejected individually without blocking unrelated valid tasks.
- Policy failures can either fail closed or use a bounded fail-open path.
- GPU and total concurrency limits continue to apply during fail-open operation.
- Duplicate identities and dangerous plugin-name collisions are handled conservatively.

The fail-open path selects valid tasks in queue order. See **policy.go**.

This fallback means:

```
The policy engine failed
-> continue conservatively using FIFO
```

It does **not** mean:

```
DINO failed
-> launch CNN
```

There are no retries, model alternatives, checkpoints, replicas, migrations, or automatic recovery actions.

## 5. THOR/HALO camera orchestration prototype

The current working tree also contains an **untracked** pkg/orchestration package. Because it is untracked, it is not yet part of the committed/documented MVP.

**What already exists in that prototype:**

- A generic local-camera interface.
- A versioned `halo.capture/v1` request/response contract.
- A THOR-side remote HALO camera adapter.
- A HALO-side camera endpoint.
- Concurrent capture of the THOR and HALO images.
- Correlation using the same capture request ID.
- Validation of camera identity, media type, image size, timestamp, and request ID.
- A maximum allowed timestamp skew between the two images.
- Serialization so two sessions cannot simultaneously use the same physical camera pair.
- A generic `PairAnalyzer` boundary receiving the validated image pair.

See **types.go**, **coordinator.go**, and **halo.go**.

Its default bounds are:

- 15-second capture timeout
- 2-second maximum capture skew
- 8 MiB maximum image size

What it does not contain:

- An actual camera driver
- MQTT implementation
- Meshtastic implementation
- Image compression or fragmentation
- WAN transport
- Mortimus
- CNN, DINO, JEPA, or another analyzer
- Analyzer results or smoke alerts
- Integration with the scheduler binary

So it establishes a clean camera-pairing boundary, but the concrete transport and AI implementation still need to plug into its interfaces.

## 6. Offline tools

### **policyctl**

`policyctl` runs the policy without SAGE or Kubernetes.

It can:

- Validate policy YAML
- Read a JSON snapshot

- Calculate a decision
- Produce explained JSON output

It has no network client. See **main.go**.

The supplied snapshot compares an urgent GPU smoke detector against a routine image sampler: **urgent-vs-routine.json**.

## **chaoslab**

chaoslab performs deterministic sensitivity analysis.

It can perturb:

- Priority
- Arrival time
- Deadline
- Estimated runtime
- Maximum queue latency
- Success rate
- Available resources

It reruns the policy and compares the selected sets using Jaccard distance. See **experiment.go**.

It does not inject faults into a real cluster.

## **7. SAGE integration binary**

The integration module pins a specific Waggle edge-scheduler commit and builds a replacement sage-nodescheduler executable.

**What already exists:**

- Loading of Waggle runtime configuration
- Loading of the resilient policy configuration
- Construction of the upstream NodeScheduler
- Injection of this repository's policy
- Offline --validate-config mode
- Rejection of unsafe live simulation modes
- Rejection of empty or inherited default RabbitMQ credentials

See **waggle-nodescheduler/main.go**.

The documented status is still: **prepared but not deployed or connected to a SAGE node**.

## **8. Packaging and deployment**

**What already exists:**

- A multi-stage Docker build.
- A static sage-nodescheduler binary.
- A minimal distroless, non-root runtime image.

- Example ConfigMap.
- Example Kubernetes Deployment patch.
- Manual controlled-restart instructions.

See **Dockerfile** and **deployment README**.

The manifests are examples only. The repository never applies them automatically.

Before deployment, the project still needs confirmation of the real:

- Namespace
- Deployment and container names
- SAGE/Waggle commit
- k3s version
- CPU architectures
- Service account
- Image digest
- RabbitMQ secrets
- Network exposure

## 9. Tests and validation

The repository contains:

- Policy ranking and admission unit tests
- Determinism and permutation tests
- Fuzz testing
- Configuration validation tests
- Resource parser tests
- Adapter queue-safety tests
- Fail-open/fail-closed tests
- CLI tests
- Chaos sensitivity tests
- SAGE job-schema tests
- Camera-pair orchestration tests in the untracked prototype
- Optional KWOK Kubernetes fixtures

The KWOK fixture simulates CPU and GPU nodes and validates Kubernetes placement, but it does not run actual containers, GPUs, cameras, RabbitMQ, or the scheduler. See **test/kwok/README.md**.

The **Makefile** combines formatting, tests, race tests, vetting, builds, configuration validation, and examples.

I could not execute the tests in this inspection because the current shell does not have the go executable installed.

## 10. Important remaining limitations

The biggest limitations are:

- The upstream Waggle queue removes some tasks using plugin name instead of full identity.
- The queue cannot currently be snapshotted atomically.
- The local scheduling REST endpoint in the pinned Waggle revision creates incomplete runtime objects.
- Pod-creation failures are not reliably requeued upstream.
- Queue age exists only in memory and resets after restart.
- Available capacity is not currently trustworthy.
- There are no NodeScheduler Prometheus metrics.
- There is no learned prediction or runtime success feedback.
- There is no preemption, retry budget, checkpointing, replication, or migration.
- There is no AI-model degradation or alternative-model selection.
- The Waggle dependency's licensing must be confirmed before distributing a compiled binary; see **NOTICE**.

## Mapping to your diagram

For the architecture you showed:

- **SAGE scheduler box:** implemented as the generic urgent/resilient scheduling policy.
- **THOR <-> HALO image request:** partially represented by the new untracked orchestration prototype.
- **Camera correlation and paired-image validation:** implemented in that prototype.
- **MQTT/Meshtastic transport:** not implemented.
- **Mortimus smoke detector:** not implemented here.
- **CNN/DINO/JEPA inference:** not implemented here.
- **Choosing among CNN/DINO/JEPA:** not implemented.
- **Publishing alerts to WAN-THOR/Beehive:** not implemented by this repo.
- **Actual SAGE deployment:** prepared, but not performed.

The cleanest description is: **the repo currently provides the scheduling brain, offline policy laboratory, SAGE adapter, and an uncommitted camera-pair orchestration boundary. The transport, smoke-detection application, model cascade, and operational deployment remain separate work.**